Final Project Part 1: **Optimization in High-Performance Computing**

Anushka Nanaware

STUDENT ID: 005042356

Department of Computer Science

University of the Cumberlands

MSCS532 - M80 Algorithms and Data Structures

Dr. Micahel Solomon

April 5, 2026

GITHUB LINK: https://github.com/ananaware/MSCS532-Project_HPC-Data-Locality-Optimization/tree/main

High-performance computing (HPC) is used to solve large and complex problems efficiently. In these systems, performance depends not only on the algorithm used but also on how data is stored and accessed. Even if an algorithm is efficient, poor memory usage can slow down execution significantly.

The research paper used in this assignment explains that many performance issues in HPC systems come from inefficient data access patterns, especially when programs do not use memory efficiently (Hassan et al., 2023). This shows that optimization is not only about improving algorithms, but also about improving how data is handled in memory. In this project, I focus on **data locality optimization**, which improves how data is accessed in memory. The goal is to demonstrate how changing data access patterns can significantly improve performance, even when the overall task remains the same.

Optimization Techniques in HPC

The research study identifies several optimization challenges in HPC systems. One of the major issues is inefficient use of memory and poor alignment with hardware architecture.

Programs that do not consider how data is stored in memory often suffer from slow performance due to cache misses (Hassan et al., 2023).

Data locality is an important concept in this context. It refers to accessing data that is stored close together in memory. When data is accessed sequentially, it improves cache usage and reduces memory access time.

From an algorithm perspective, performance is not only determined by time complexity but also by how efficiently memory is used. As discussed in *Introduction to Algorithms*, practical performance depends on both algorithm design and how data is accessed during execution (Cormen et al., 2022).

Selected Optimization Technique: Data Locality

The optimization technique selected for this project is **data locality optimization**. This technique was chosen because:

- It is clearly discussed in the research paper
- It directly impacts performance in HPC systems
- It is simple to implement and demonstrate

Data locality optimization works by improving how data is accessed in memory. Modern processors use cache memory to store frequently accessed data. If data is accessed sequentially (contiguous access), the processor can reuse cached data efficiently. However, if data is accessed randomly, it leads to frequent cache misses, which slows down execution (Hassan et al., 2023).

Strengths:

- Improves execution speed
- Reduces memory access time
- Easy to apply in many programs

Weaknesses:

- May require changes to data structures
- Benefits depend on hardware
- Not always obvious in complex systems

Implementation of the Optimization

To demonstrate data locality optimization, I implemented a small Python program with two versions:

Version 1: Non-Contiguous Access (Cache-Unfriendly)

In this version, data is accessed in a random order. This simulates poor memory usage and leads to inefficient cache utilization.

Version 2: Contiguous Access (Cache-Friendly)

In this version, data is accessed sequentially. This improves cache usage and allows faster execution.

Both versions perform the same operation (summing elements in a large dataset). The only difference is the order in which data is accessed.

```
C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>notepad notes\pape
r_notes.txt

C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>notepad src\data_l
ocality_demo.py

C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>python src\data_lo
cality_demo.py
Running non-contiguous access...
Non-contiguous time: 10.8872 seconds

Running contiguous access...
Contiguous time: 0.3737 seconds

C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>
```

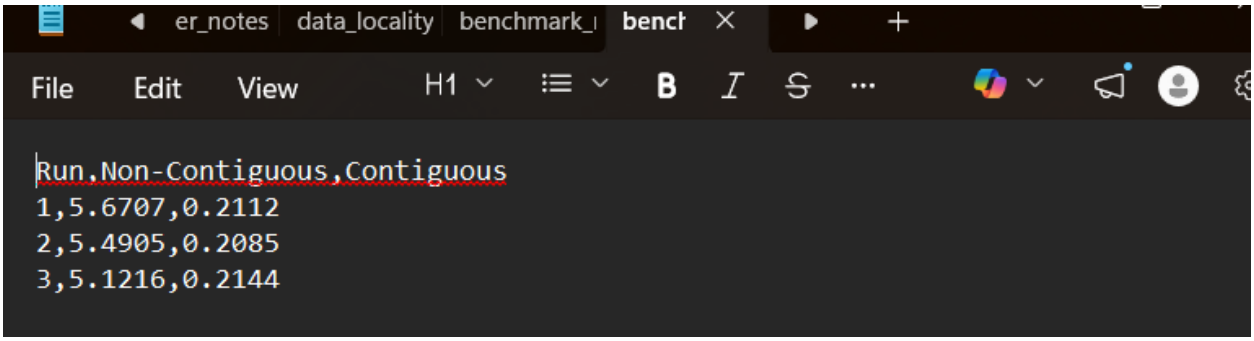
Experimental Results and Analysis

To ensure reliable results, the experiment was repeated multiple times using a benchmark script. **Observed Results:**

- Non-contiguous access: approximately **5.1 to 5.6 seconds**
- Contiguous access: approximately **0.20 to 0.21 seconds**

This shows a performance improvement of about **25 times faster** when using contiguous access.

```
C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>notepad src\benchm
ark_runner.py

C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>python src\benchma
rk_runner.py

Run 1
Non-contiguous: 5.6707 sec
Contiguous: 0.2112 sec

Run 2
Non-contiguous: 5.4905 sec
Contiguous: 0.2085 sec

Run 3
Non-contiguous: 5.1216 sec
Contiguous: 0.2144 sec

Results saved to results/benchmark_results.txt

C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>type results\bench
mark_results.txt
Run,Non-Contiguous,Contiguous
1,5.6707,0.2112
2,5.4905,0.2085
3,5.1216,0.2144

C:\Users\anush\Documents\MSCS532_HPC_Optimization_Project>
```



Analysis

The results clearly show that accessing data sequentially is much faster than accessing it randomly. This happens because:

- Sequential access improves cache usage
- Random access causes cache misses
- Cache misses increase memory access time

These results match the findings from the research paper, which emphasizes the importance of optimizing memory access patterns in HPC systems (Hassan et al., 2023).

Challenges

During the implementation, some challenges were encountered:

- The initial dataset size was too large and caused long execution time
- Random access added extra overhead due to shuffling

- Ensuring fair comparison between both approaches required multiple runs

These challenges were resolved by adjusting dataset size and using consistent benchmarking.

This project helped me understand that:

- Performance optimization is not only about algorithms
- Memory access patterns have a huge impact on execution time
- Small changes in implementation can lead to large improvements

The results observed in this project were consistent with both theoretical expectations and the research study.

In conclusion, this project demonstrated how data locality optimization improves performance in high-performance computing systems. By simply changing how data is accessed, a significant improvement in execution time was achieved.

This shows that efficient use of memory is just as important as choosing the right algorithm. Data locality is a simple but powerful technique that can be applied to improve performance in many real-world applications.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hassan, F., et al. (2023). *An empirical study of high-performance computing optimization techniques*. https://foyzulhassan.github.io/files/MSR23_HPC.pdf